Project Report: AI Language Detector & Translator

Introduction & Problem Statement:

In our highly connected world, tools that bridge cross-cultural communication barriers are essential. This project presents an end-to-end Natural Language Processing (NLP) web application capable of real-time language detection and translation. Powered by a custom Multinomial Naive Bayes classifier and TF-IDF vectorization, the system accurately identifies text across multiple languages and integrates with deep learning APIs to provide instant English translations.

Methodology:

This project utilizes a standard Machine Learning pipeline:

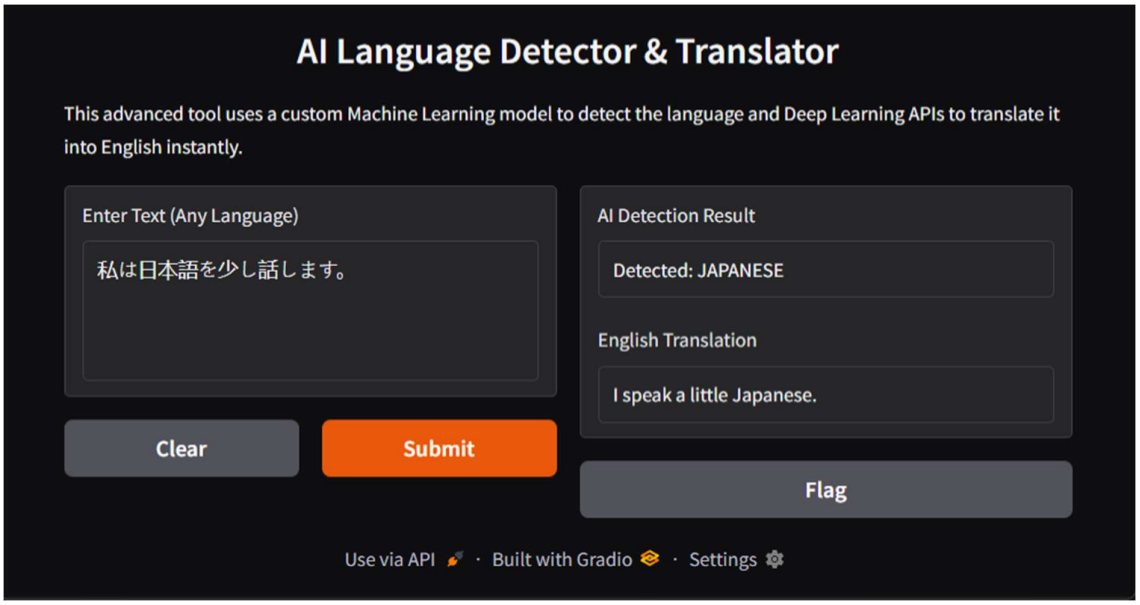
- **Data Preparation:** Text samples from multiple languages are merged using the pandas library, cleaned of exact duplicates to prevent model overfitting, and split into an 80/20 training and testing set.
- **Feature Extraction (TF-IDF):** To handle overlapping vocabulary, the text is processed using TfidfVectorizer with character n-grams (3 to 5 letters). This normalizes the data and trains the model to recognize unique letter combinations rather than relying on exact word matches.
- **Model Training:** A Multinomial Naive Bayes classifier (MultinomialNB) calculates the probability of specific letter patterns belonging to a given language.
- **Model Persistence:** The trained pipeline is serialized and exported as a .pkl file using joblib for low-latency inference without the need to retrain.

Implementation & Technologies Used:

- **Programming Language:** Python
- **Machine Learning:** Scikit-Learn (TfidfVectorizer, MultinomialNB, Pipeline)
- **Data Manipulation:** Pandas, NumPy
- **User Interface:** Gradio (for creating the interactive web application)

- **External APIs:** deep_translator (for routing detected text to Google Translate)

Output:



Conclusion:

This project successfully demonstrates a full-stack ML workflow. The final application provides an intuitive, user-friendly interface where users can type in any language, instantly see the AI's detection results, and receive an accurate English translation. It highlights the importance of proper data vectorization and feature engineering in solving real-world NLP problems.

Future Scope

Future improvements for this application could include:

1. Expanding the dataset to include non-Latin scripts (e.g., Arabic, Hindi, Mandarin) to test the vectorizer's robustness.
2. Implementing an "Audio Output" feature using a Text-to-Speech (TTS) library so users can hear the translation.
3. Deploying the application permanently to a cloud-hosted platform like Hugging Face Spaces or AWS.